# UT4 - OPTIMIZACIÓN Y DOCUMENTACIÓN

Entornos de Desarrollo - DW1A

*Roha Magdaleno, Borja Osoro*

# REFACTORIZACIÓN

## *¿Para qué sirve refactorizar?*

La refactorización es el proceso de mejorar la estructura interna del código sin cambiar su comportamiento externo. El programa hace lo mismo pero el código queda más limpio, legible, mantenible y menos propenso a errores.

En pocas palabras: ordenar y pulir el código sin romper nada.

## *ANTES*

Código poco legible, con nombres poco claros y un método difícil de entender

```java
package refactorizacion;

public class Factura {
    public double c(double p, int c, boolean d) {
        double r = p * c;
        if (d) {
            r = r - (r * 0.1);
        }
        return r;
    }
}
```

## *DESPUÉS*

El resultado es el mismo pero el código es mucho más claro y comprensible

```java
package refactorizacion;

public class Factura {

    public double calcularTotal(double precioUnitario, int cantidad, boolean tieneDescuento) {
        double total = precioUnitario * cantidad;

        if (tieneDescuento) {
            total = total - (total * 0.1);
        }

        return total;
    }
}
```

# LIMITACIONES

- No agrega funcionalidad nueva
- No corrige errores de lógica
- Depende mucho de tener pruebas (tests)
- Consume tiempo y recursos
- No arregla una mala arquitectura de raíz
- Riesgo de sobre-refactorización
- Limitada en sistemas muy antiguos o críticos

# PATRONES DE REFACTORIZACIÓN

***Beneficios: legibilidad, mantenibilidad, calidad del diseño y patrones de refactorización***

- Renombrar variables o funciones con nombre más claros
- Dividir funciones o métodos muy largos en métodos más pequeños
- Eliminar código duplicado
- Simplificar condicionales complejos
- Reorganizar clases y métodos
- Mejorar la estructura sin cambiar la lógica

**1.Ejemplo de como renombrar variables**

```java
package refactorizacion;

public class Factura {

    // Antes
int x;

    // Después
int numeroDeProductos;


}
```

**1.Ejemplo de como extraer métodos**

***ANTES***

```java
package refactorizacion;

    public class ExtraerMetodoAntes {

    public static void main(String[] args) {
        imprimirFactura(50, 2);
    }

    public static void imprimirFactura(double precio, int cantidad) {
        double total = precio * cantidad;
        total = total + (total * 0.21);
        System.out.println("Total: " + total);
    }
}
```

***DESPUÉS***

```java
package refactorizacion;

    public class ExtraerMetodoDespues {

    private static final double IVA = 0.21;

    public static void main(String[] args) {
        imprimirFactura(50, 2);
    }

    public static void imprimirFactura(double precio, int cantidad) {
        double total = calcularTotalConIVA(precio, cantidad);
        System.out.println("Total: " + total);
    }

    public static double calcularTotalConIVA(double precio, int cantidad) {
        double subtotal = precio * cantidad;
        return subtotal + (subtotal * IVA);
    }
}
```

# 1.Ejemplo de como mover métodos o clases

```java
package refactorizacion;

    public class PedidoApp {

    public static void main(String[] args) {
        Pedido pedido = new Pedido(20, 3);
        double total = pedido.calcularTotal();
        System.out.println("Total del pedido: " + total);
    }
}
```

```java
package refactorizacion;

    public class PedidoApp {

    public static void main(String[] args) {
        double total = calcularTotal(20, 3);
        System.out.println("Total del pedido: " + total);
    }

    // Este método no debería estar aquí
    public static double calcularTotal(double precioUnitario, int cantidad) {
        double subtotal = precioUnitario * cantidad;
        return subtotal + (subtotal * 0.21);
    }
}
```

Se utiliza cuando un método o clase está ubicada en un lugar que no es el más adecuado.

Moverlos a la clase correcta mejora la cohesión (cada clase hace lo que le corresponde) y reduce el acoplamiento, haciendo el diseño más limpio y lógico.

Ejemplo visual en NetBeans con varias capturas:

```java
package refactorizacion;

class Pedido {

    Pedido(int i, int i0) {
        throw new UnsupportedOperationException("Not supported yet.");
    }

    double calcularTotal() {
        throw new UnsupportedOperationException("Not supported yet.");
    }

    private static final double IVA = 0.21;

    private double precioUnitario;
    private int cantidad;

    public Pedido(double precioUnitario, int cantidad) {
        this.precioUnitario = precioUnitario;
        this.cantidad = cantidad;
    }

    public double calcularTotal() {
        double subtotal = precioUnitario * cantidad;
        return subtotal + (subtotal * IVA);
    }

}
```

# 1.Eliminar código duplicado

Consiste en identificar bloques de código repetidos y unificarlos en un solo método o clase.
Esto evita errores futuros, ya que los cambios solo se realizan en un único lugar, y mejora la mantenibilidad y consistencia del sistema. Aquí tenemos un ejemplo.

## ANTES

```java
package refactorizacion;

    public class CodigoDuplicadoAntes {

    public static void main(String[] args) {
        mostrarTotalProductoA(10, 2);
        mostrarTotalProductoB(5, 4);
    }

    public static void mostrarTotalProductoA(double precio, int cantidad) {
        double total = precio * cantidad;
        total = total + (total * 0.21);
        System.out.println("Total producto A: " + total);
    }

    public static void mostrarTotalProductoB(double precio, int cantidad) {
        double total = precio * cantidad;
        total = total + (total * 0.21);
        System.out.println("Total producto B: " + total);
    }
}
```

## DESPUÉS

```java
package refactorizacion;

    public class CodigoDuplicadoDespues {

    private static final double IVA = 0.21;

    public static void main(String[] args) {
        mostrarTotal("Producto A", 10, 2);
        mostrarTotal("Producto B", 5, 4);
    }

    public static void mostrarTotal(String nombreProducto, double precio, int cantidad) {
        double total = calcularTotalConIVA(precio, cantidad);
        System.out.println("Total " + nombreProducto + ": " + total);
    }

    public static double calcularTotalConIVA(double precio, int cantidad) {
        double subtotal = precio * cantidad;
        return subtotal + (subtotal * IVA);
    }
}
```

# 1. Ejemplo de como simplificar condicionales

## ANTES

```java
package refactorizacion;

public class CondicionalAntes {

    public static void main(String[] args) {
        System.out.println(estaActivo(true));
    }

    public static boolean estaActivo(boolean activo) {
        if (activo == true) {
            return true;
        } else {
            return false;
        }
    }
}
```

## DESPUÉS

```java
package refactorizacion;

    public class CondicionalDespues {

    public static void main(String[] args) {
        System.out.println(estaActivo(true));
    }

    public static boolean estaActivo(boolean activo) {
        return activo;
    }
}
```

Busca reducir la complejidad de estructuras if, else o switch demasiado largas o confusas.
Se logra usando técnicas como métodos específicos, polimorfismo o expresiones más claras, lo que hace el código más fácil de entender, leer y modificar.

# REFACTORIZACIÓN Y PRUEBAS

- La refactorización debe apoyarse en pruebas automatizadas para garantizar que el comportamiento del software no cambie. Estas pruebas, sobretodo las **pruebas unitarias**, permiten verificar que el código siga funcionando correctamente después de cada modificación, reduciendo el riesgo de de introducir errores y facilitando mejoras seguras en la calidad del código.

# HERRAMIENTAS DE AYUDA

### *NetBeans*

Incluye potentes herramientas de refactorización que permiten:

- Renombrar clases, métodos y variables, actualizando automáticamente todas sus referencias.
- Extraer métodos para mejorar la legibilidad  y reutilización del código.
- Mover clases o paquetes sin romper dependencias.
- Detectar errores y sugerir mejoras en el código.

### *Visual Studio Code*

Mediante extensiones y soportes de lenguajes, ofrece funciones de refactorización como:

- Renombrado automático de símbolos en todo el proyecto.
- Extracción de métodos, variables o constantes.
- Organización y limpieza de importaciones.
- Sugerencias de mejoras en tiempo real gracias a analizadores de código.

# ¿QUE ES EL CONTROL DE VERSIONES?

- El control de versiones permite gestionar los cambios realizados en el código fuente de un proyecto.
- Facilita el trabajo en equipo, el seguimiento de modificaciones y la recuperación de versiones anteriores.
- Es fundamental en desarrollos colaborativos y proyectos reales.

# DESARROLLOS COLECTIVOS

- Varios desarrolladores trabajan sobre el mismo proyecto.
- Cada cambio queda registrado con autor y fecha.
- Permite detectar errores y volver atrás si algo falla.
- Evita sobrescribir el trabajo de otros compañeros.

# HERRAMIENTAS DE CONTROL DE VERSIONES

Existen diferentes herramientas de control de versiones, cada una con características propias, aunque algunas son más utilizadas que otras en el entorno profesional.

### *GIT*

- Sistema de control de versiones distribuido.
- Muy rápido y flexible.
- El más utilizado actualmente en proyectos profesionales.
- Se integra con plataformas como GitHub, GitLab o Bitbucket.

### *SVG*

- Sistema centralizado.
- Todos los cambios dependen de un servidor central.
- Más sencillo, pero menos flexible que Git.

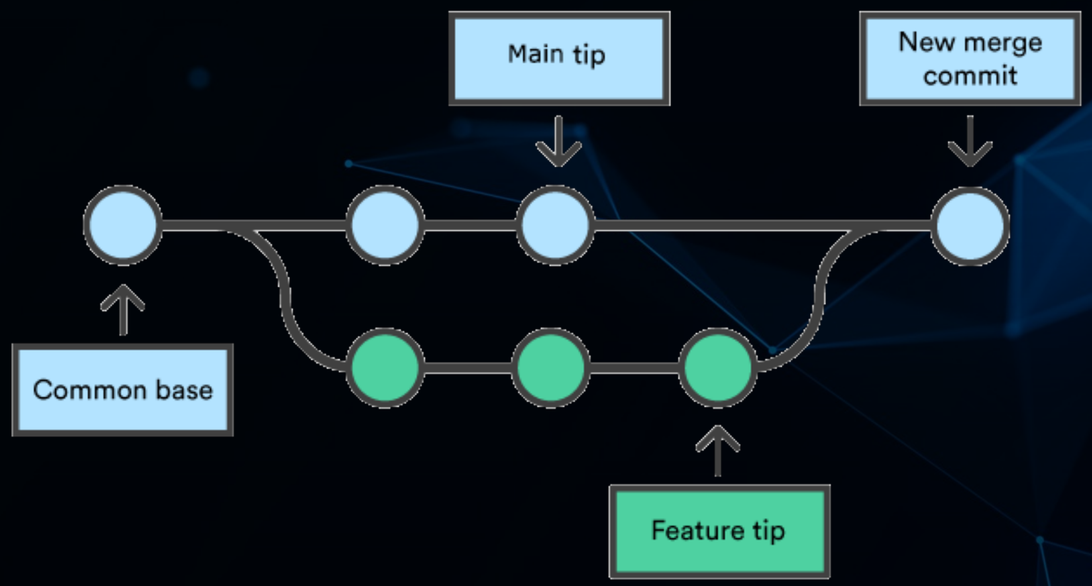
### *MERCURIAL*

- Sistema distribuido, similar a Git.
- Interfaz más simple.
- Menos usado, pero muy estable y eficiente.

# CARACTERÍSTICAS PRINCIPALES

Aunque existen varios sistemas de control de versiones, Git es el más utilizado actualmente, por lo que nos centramos en sus características principales.

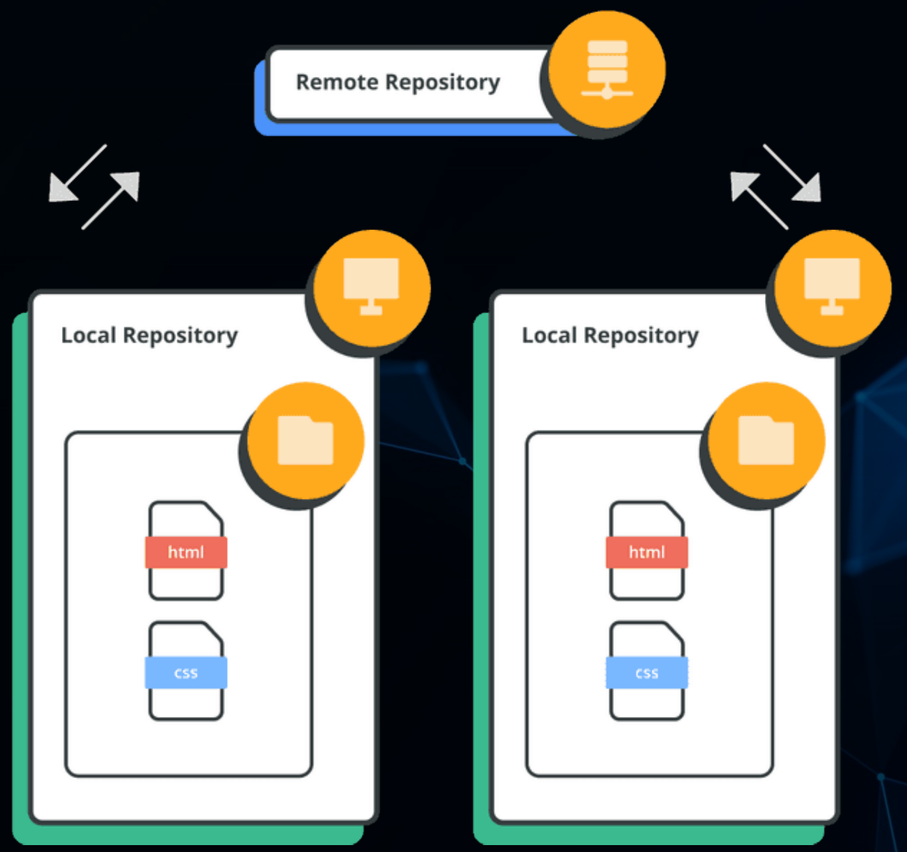
- Control de versiones distribuido.
- Cada usuario tiene una copia completa del repositorio.
- Uso de ramas (branches) para trabajar en nuevas funciones.
- Fusiones (merge) para integrar cambios.
- Permite trabajar sin conexión y sincronizar después.



# ESTRUCTURA DEL SISTEMA DE VERSIONES

El funcionamiento de un sistema de control de versiones se basa en una estructura clara que permite trabajar de forma segura y organizada.

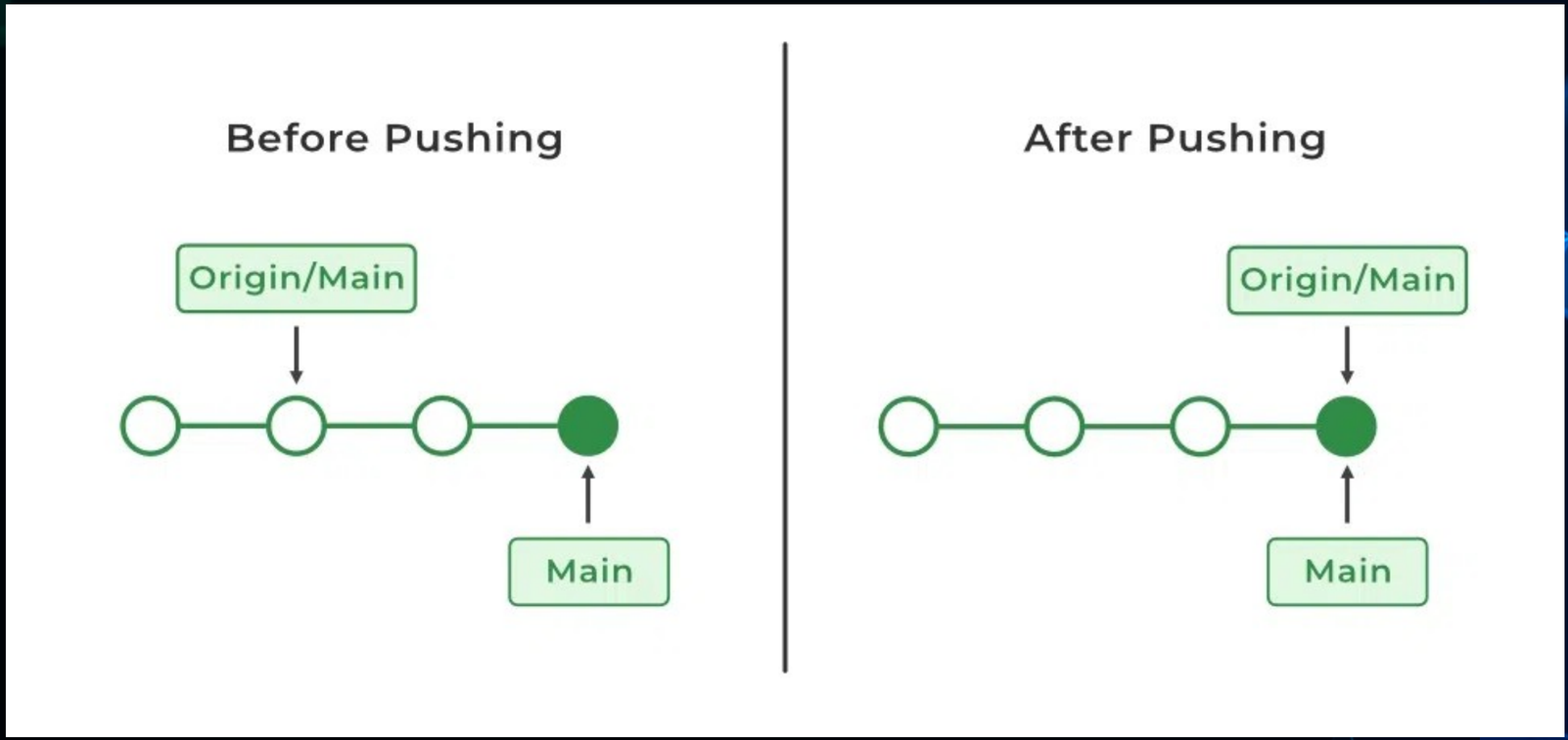
- Modelo cliente/servidor.
- Repositorio remoto: almacena el proyecto principal.
- Repositorio local: copia completa en el ordenador del desarrollador.
- Comunicación constante entre local y remoto.
- Mayor seguridad frente a pérdidas de datos.



# OPERACIONES BÁSICAS(I)

Para trabajar con un sistema de control de versiones es necesario conocer una serie de operaciones básicas que se utilizan de forma habitual.

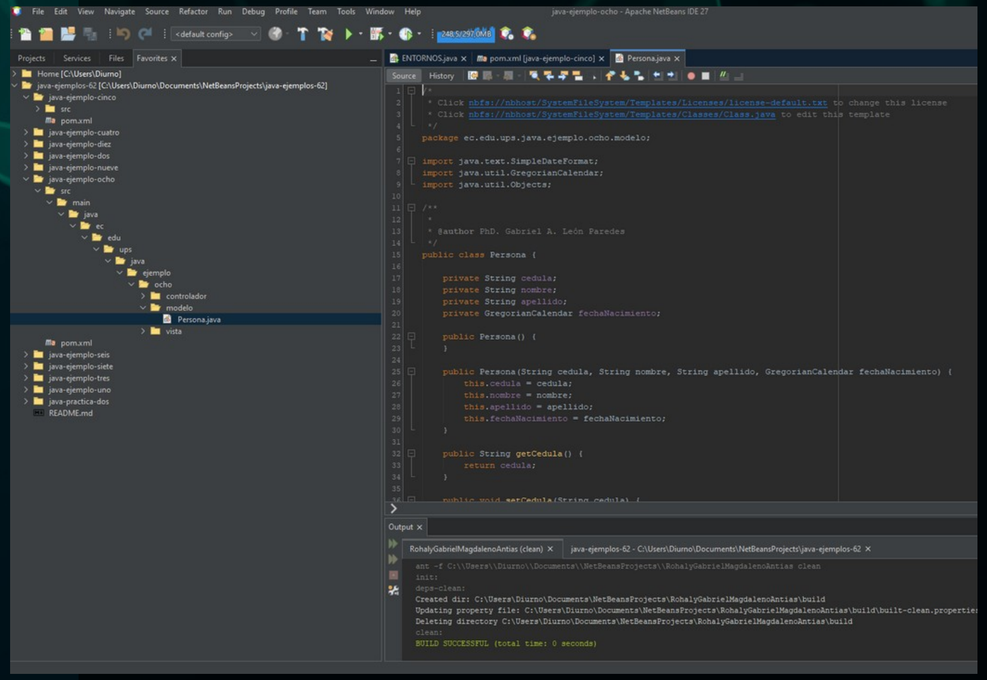
- Clonar: descargar el proyecto desde el repositorio remoto.
- Modificar: editar archivos en local.
- Commit: guardar los cambios con un mensaje descriptivo.
- Push: subir los cambios al repositorio remoto.



# OPERACIONES BÁSICAS(I)

CLONAR UN REPOSITORIO (Git):

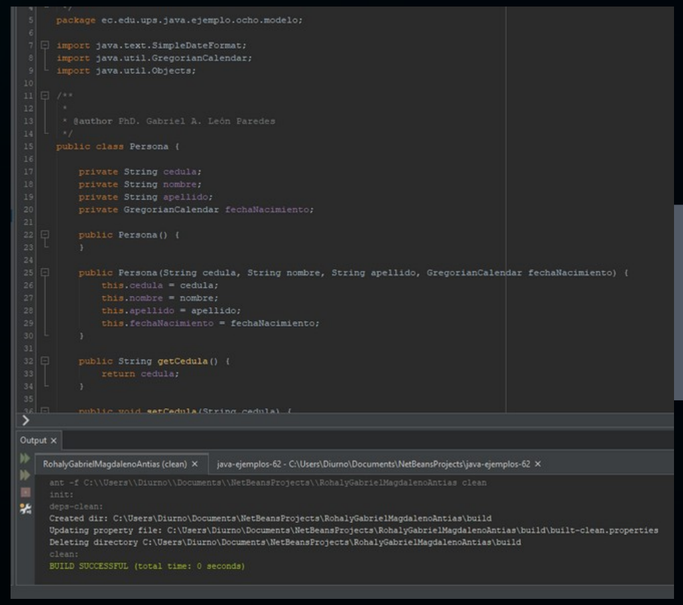
Descarga inicial del proyecto

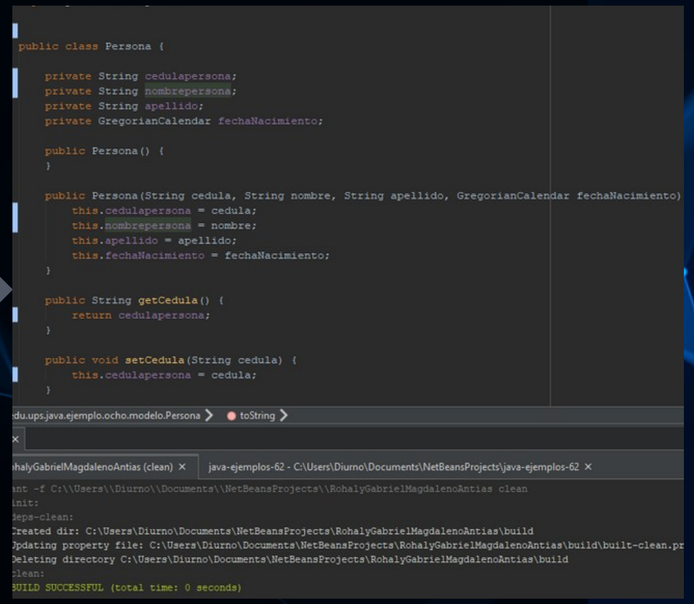



MODIFICAR UN ARCHIVO:

Trabajo en local

En este ejemplo he cambiado el nombre de la variable cedula por cedula persona y la variable nombre por nombrepersona

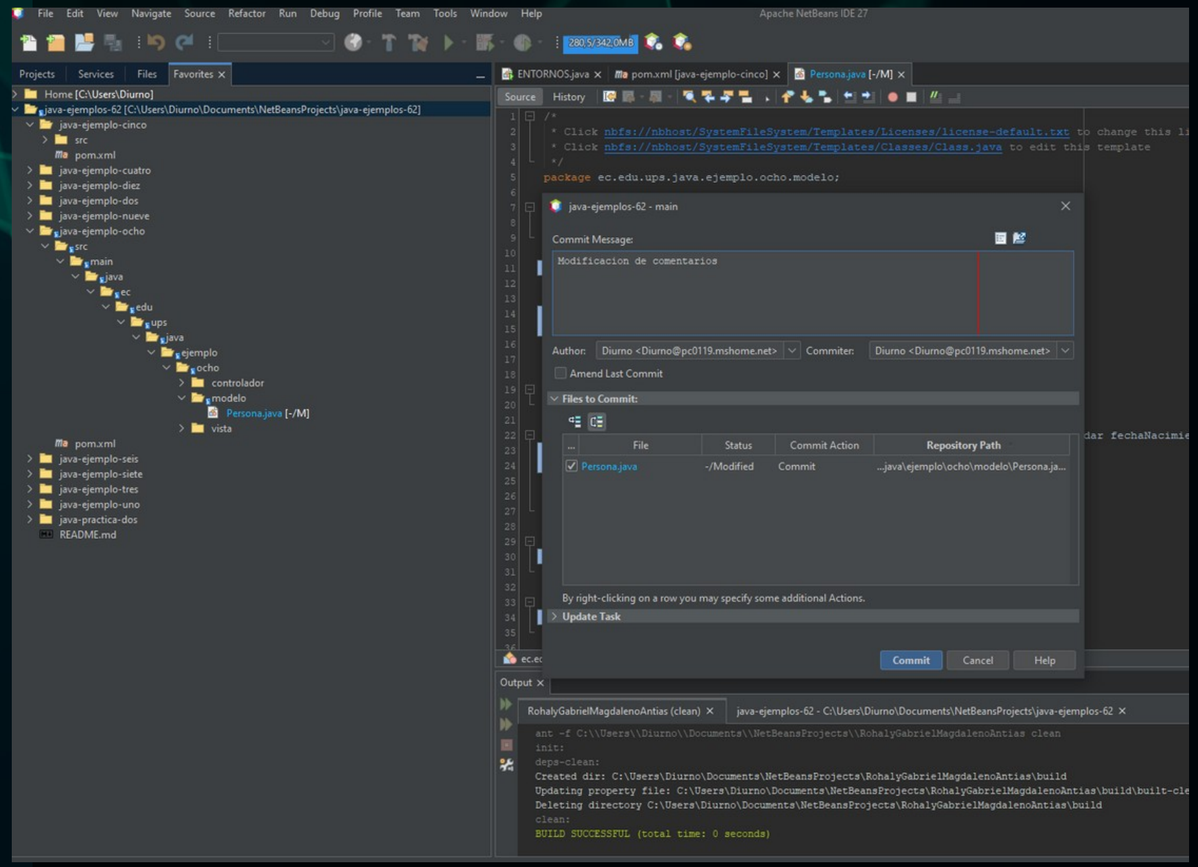


# OPERACIONES BÁSICAS(I)

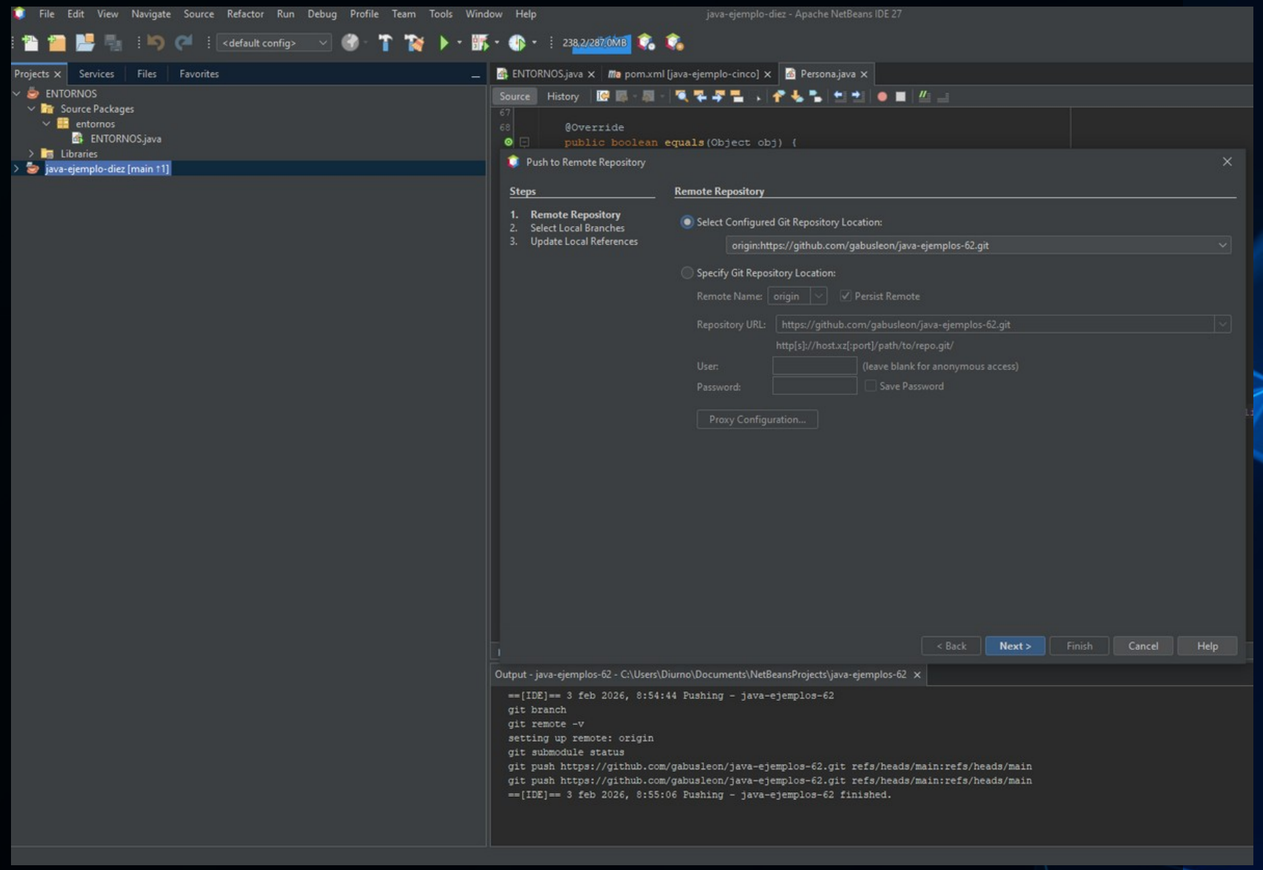
COMMIT DE CAMBIOS:

Guardado de cambios




PUSH AL REPOSITORIO:

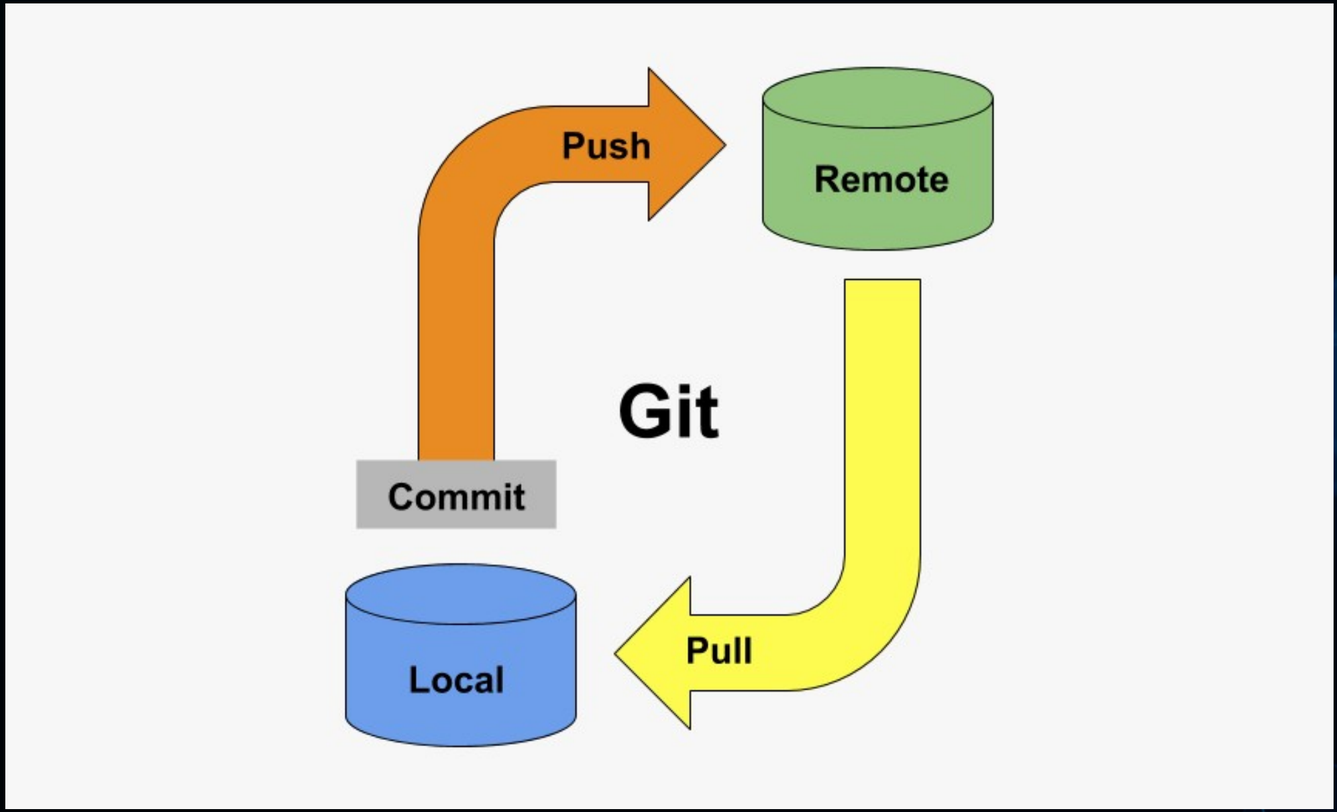
Subida de cambios



# OPERACIONES BÁSICAS(II)

Para trabajar con un sistema de control de versiones es necesario conocer una serie de operaciones básicas que se utilizan de forma habitual.

- Pull: actualizar el proyecto local con los cambios remotos.
- Historial: consultar commits anteriores.
- Comparación: ver diferencias entre versiones.
- Restauración: volver a un estado anterior del proyecto.
- Conflictos: resolver cambios incompatibles entre archivos.



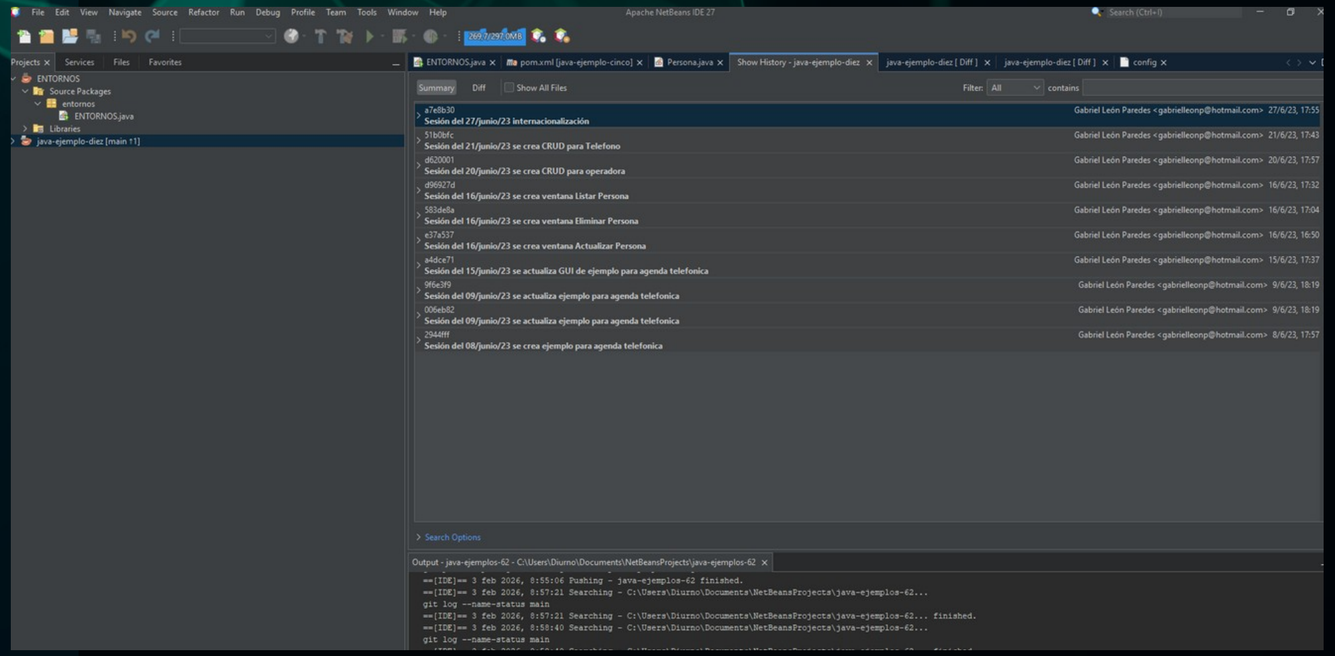
# OPERACIONES BÁSICAS(II)

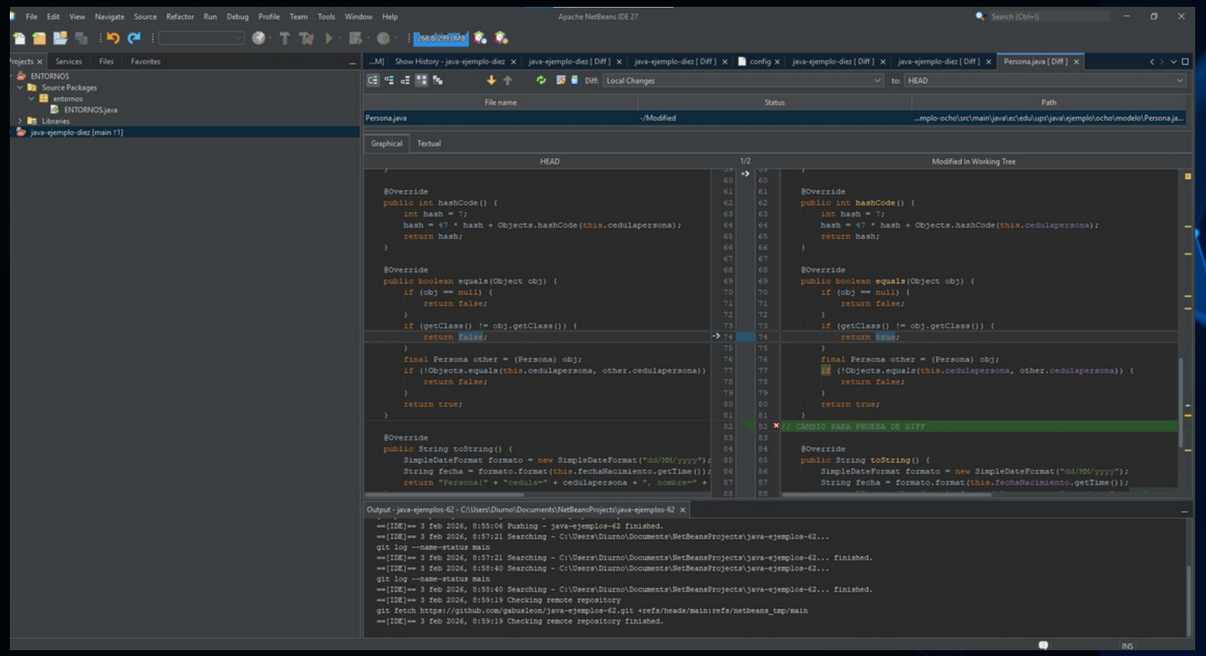
HISTORIAL DE COMMITS:

Seguimiento de versiones

DIFF:

Comparación entre versiones



# VENTAJAS DEL CONTROL DE VERSIONES

El uso de sistemas de control de versiones aporta numerosas ventajas en el desarrollo de software, especialmente en proyectos colaborativos.

- Mayor organización y seguridad del código.
- Facilita el trabajo en equipo.
- Permite experimentar sin riesgo.
- Mejora el mantenimiento del software.
- Muy utilizado en entornos profesionales.

# ¿POR QUÉ DOCUMENTAR EL SOFTWARE?

La documentación es una parte fundamental del desarrollo de software, ya que facilita su comprensión, mantenimiento y evolución.

- Permite comprender el funcionamiento del programa.
- Facilita el mantenimiento y futuras modificaciones.
- Ayuda a nuevos desarrolladores a integrarse en el proyecto.
- Reduce errores y mejora la calidad del software.

# COMENTARIOS EN EL CÓDIGO

Los comentarios son una herramienta básica para explicar el código y facilitar su comprensión.

- Los comentarios explican partes complejas del código.
- Deben ser claros, breves y útiles.
- No deben repetir lo que ya se entiende leyendo el código.
- Un exceso de comentarios puede dificultar la lectura.

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.ups.java.ejemplo.ocho.modelo;

import java.text.SimpleDateFormat;
import java.util.GregorianCalendar;
import java.util.Objects;


public class Persona {

    private String cedulapersona;
    private String nombrepersona;
    private String apellido;
    private GregorianCalendar fechaNacimiento;
//CONSTRUCTOR POR DEFECTO
    public Persona() {
    }

    public Persona(String cedula, String nombre, String apellido, GregorianCalendar fechaNacimiento) {
        this.cedulapersona = cedula;
        this.nombrepersona = nombre;
        this.apellido = apellido;
        this.fechaNacimiento = fechaNacimiento;
    }

    public String getCedula() {
        return cedulapersona;
    }

    public void setCedula(String cedula) {
        this.cedulapersona = cedula;
    }

    public String getNombre() {
        return nombrepersona;
    }

    public void setNombre(String nombre) {
        this.nombrepersona = nombre;
    }

    public String getApellido() {
        return apellido;
    }
```

# DOCUMENTACIÓN AUTOMÁTICA

Existen herramientas que permiten generar documentación automáticamente a partir del propio código fuente.

- Genera documentación a partir del propio código.
- Mantiene la documentación actualizada.
- Reduce errores y ahorra tiempo.
- Herramientas más usadas:
  1. JavaDoc (Java)
  2. Doxygen (C, C++, Java...)
  3. Docstrings (Python)

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package ec.edu.ups.java.ejemplo.ocho.modelo;

import java.text.SimpleDateFormat;
import java.util.GregorianCalendar;
import java.util.Objects;

/**
 * Clase que gestiona usuarios
 */

public class Persona {

    private String cedulapersona;
    private String nombrepersona;
    private String apellido;
    private GregorianCalendar fechaNacimiento;
//CONSTRUCTOR POR DEFECTO

    /**
     *
     */
    public Persona() {
    }

    /**
     *
     * @param cedula
     * @param nombre
     * @param apellido
     * @param fechaNacimiento
     */
    public Persona(String cedula, String nombre, String apellido, GregorianCalendar fechaNacimiento) {
        this.cedulapersona = cedula;
        this.nombrepersona = nombre;
        this.apellido = apellido;
        this.fechaNacimiento = fechaNacimiento;
    }
```

# OTRAS FORMAS DE DOCUMENTACIÓN

## README

- Es el primer documento que se consulta en un proyecto de software.
- Explica de forma general:
  - El objetivo del proyecto.
  - Cómo instalarlo y ejecutarlo.
  - Tecnologías utilizadas.
- Es fundamental en repositorios Git (GitHub, GitLab).

## WIKIS

- Permiten una documentación colaborativa y estructurada.
- Se actualizan fácilmente conforme evoluciona el software.
- Muy usadas en proyectos grandes y equipos distribuidos.
- Ejemplos: GitHub Wiki, GitLab Wiki, Confluence.

## MANUALES TECNICOS

- Documentos más detallados y formales.
- Explican el funcionamiento interno del sistema.
- Incluyen configuración, arquitectura y casos de uso.
- Orientados a desarrolladores o personal técnico.

## DIAGRAMAS UML

- Representan visualmente la estructura del software.
- Facilitan la comprensión del diseño del sistema.
- Tipos más comunes:
- Diagrama de clases.
- Diagrama de casos de uso.
- Diagrama de secuencia.
- Muy útiles para planificación y mantenimiento.

# EJEMPLOS DE OTRAS FORMAS DE DOCUMENTACIÓN

***README***

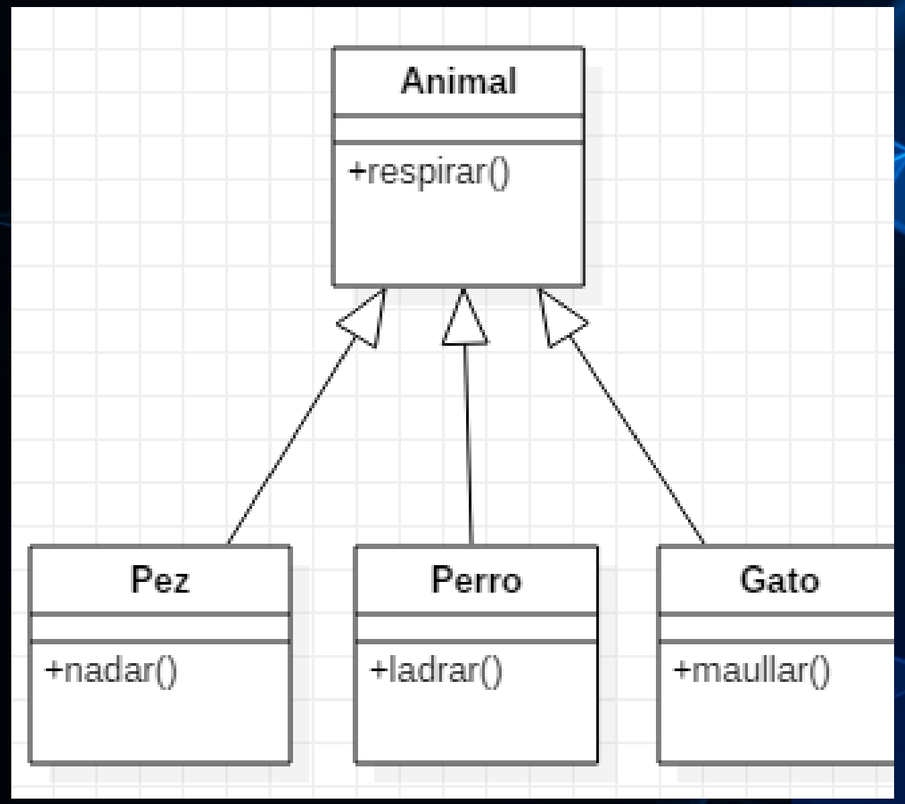
```go
func main() {
    app := fiber.New()

    // GET /api/register
    app.Get("/api/*", func(c *fiber.Ctx) error {
        msg := fmt.Sprintf("👋 %s", c.Params("*"))
        return c.SendString(msg) // => 👋 register
    }).Name("api")

    data, _ := json.MarshalIndent(app.GetRoute("api"), "", "  ")
    fmt.Print(string(data))
    // Prints:
    // {
    //     "method": "GET",
    //     "name": "api",
    //     "path": "/api/*",
    //     "params": [
    //       "*1"
    //     ]
    // }


    log.Fatal(app.Listen(":3000"))
}
```

***DIAGRAMA UML***



# PREGUNTAS EXAMEN

1. ¿Cuál es un ejemplo de herramienta de documentación automática?
a) Eclipse
b) Mercurial
c) JavaDoc ✅
d) Git

2. ¿Qué se busca evitar al refactorizar sin pruebas?
a) Que el código quede más corto
b) Que el comportamiento del programa cambie sin darse cuenta ✅
c) Que el repositorio se llene de commits
d) Que se creen ramas innecesarias

# PREGUNTAS EXAMEN

3. ¿Qué técnica es un patrón común de refactorización?
a) Eliminar el repositorio remoto
b) Extraer método ✅
c) Convertir todo el código a otro lenguaje
d) Ejecutar el programa en modo depuración

4. ¿Qué es un merge(fusión) en control de versiones?
a) Eliminar cambios antiguos del historial
b) Unir cambios de diferentes ramas o versiones ✅
c) Crear una copia local del repositorio remoto
d) Subir automáticamente todos los archivos modificados

# PREGUNTAS EXAMEN

5. ¿Qué describe mejor la estructura de un sistema de control de versiones según el documento?
a) Solo existe repositorio local
b) Modelo cliente/servidor con repositorio remoto y copias locales ✅
c) Modelo basado únicamente en carpetas compartidas
d) Solo funciona si hay conexión permanente a internet

*Entornos de Desarrollo - DW1A*

# GRACIAS

POR SU ATENCIÓN

*Roha Magdaleno, Borja Osoro*